Genomic Variant Processing

NYU Langone: Distributed Architecture via Databricks Glow

Nitin Aggarwal, Sr DSA

03/02/2026

1. Overview

This document outlines the strategic modernization of NYU Langone's genomic variant processing pipeline. By transitioning from legacy, single-node bioinformatics tools to a highly scalable, distributed cloud architecture using **Databricks, Apache Spark, and the Glow library**, NYU Langone is positioned to handle terabyte-to-petabyte scale genomic data. This new architecture ensures data reliability, accelerates processing times, and produces analytics-ready datasets for researchers and clinicians.

2. Primary Business Use Cases (The "Why")

The shift from legacy POSIX frameworks to a distributed Lakehouse architecture is driven by several critical clinical and operational use cases:

- **Precision Medicine & Oncology Insights:** The pipeline calculates critical clinical metrics such as Tumor Mutational Burden (TMB) and Microsatellite Instability (MSI), while mapping cancer subtypes via OncoTree. It automatically joins variant data with clinical annotation databases (e.g., ClinVar, VEP), directly supporting personalized treatment plans for oncology patients.
- **Population-Scale Cohort Analysis:** Legacy tools (like `bcftools` or single-node GATK) experience Out-Of-Memory (OOM) errors and severe I/O bottlenecks when processing Terabyte/Petabyte-scale data. This distributed architecture allows researchers to instantly aggregate Allele Frequencies (AF) and Allele Counts (AC) across massive patient populations without failing.
- **Holistic Patient Profiling (Genotype + Phenotype):** The Bronze-to-Silver data flow breaks down data silos by harmonizing raw genomic structures (Short Variants, Structural Variants, Copy Number Alterations, Methylation arrays) with **Clinical/EHR data** (labs, vitals, pathology). This creates a unified 360-degree view of the patient.
- **Democratization of Genomic Data for BI:** Historically, genomic data was locked in flat files (VCF/BCF) requiring complex command-line (CLI) skills to parse. By flattening these arrays into columnar Delta Lake tables, the data is made accessible to a broader audience. BI analysts, researchers, and executives can now query complex genomics using standard **ANSI SQL** (via Databricks SQL)

Figure:

Data Flow: Bronze-to-Silver pipeline

3. The Architectural Approach (Medallion Data Lakehouse)

The pipeline employs a "Medallion" data architecture (Bronze, Silver, Gold) to progressively clean, harmonize, and enrich genomic data alongside clinical records:

- **Bronze Layer (Raw):** Centralized ingestion of raw data from external labs and sequencers. This includes raw VCFs (variant calls), XML/JSON files, test metadata, and Clinical/EHR data.
- **Silver Layer (Standardized):** Data is parsed and transformed using Databricks Glow. This layer standardizes information into distinct, queryable domains: Test Metadata, Patient Data, Variant Data, and Sample/Biospecimen Data.
- **Gold Layer (Analytics):** Highly refined tables optimized for business intelligence and clinical insights, such as cohort frequencies, clinical annotations (joined with ClinVar), and sample QC metrics.

4. Technical Strategy & Innovation

To achieve scale, the engineering team addressed several traditional distributed systems bottlenecks:

- **Distributed Processing:** Replaced single-thread POSIX tools with Spark’s distributed in-memory computation, allowing horizontal scaling across massive datasets.
- **Glow Integration:** Utilized the Glow library for complex genomic operations—such as splitting multi-allelics and normalizing variants—directly within the Spark cluster.
- **External CLI Orchestration:** Implemented the "Glow Pipe Operator" to seamlessly pass distributed data through standard bioinformatics annotation tools (like VEP or Mutect) without breaking the pipeline.
- **I/O Optimization:** Deployed a custom cache-warming strategy (copying reference genomes to local NVMe storage on all executor nodes) to eliminate critical FUSE mount latencies and file-lock exceptions.

Genomic Data Pipeline: Medallion Architecture

VCF ingestion and processing framework mapped from raw storage to analytical Gold layers.

1. BRONZE VOLUMES

Raw VCF Directory
/Volumes/.../vcf/ruslan

Reference Genome
hg38.fa + .fai (Copied to /tmp)

2. DATABRICKS COMPUTATION

Read VCF
spark.read.format("vcf").load( )

Split Multiallelics
glow.transform("split_multiallelics")

Normalize Variants
glow.transform("normalize_variants")

Glow Pipe Annotator
glow.transform("pipe", cmd=["cat"])

Extract & Flatten Arrays
F.explode_outer() -> Struct flattening

3. SILVER DELTA TABLES

variants_flat
1 row per ALT allele

variant_calls
Destructured Genotypes

variant_consq...
Exploded INFO_CSQ

vcf_pipe_quarantine
Pipe rejections

vcf_ingest_file_log
Job state (RUN_ID)

4. GOLD ANALYTICS

cohort_frequencies
Aggregated AF / AC

clinical_annotations
Joined with ClinVar

sample_qc_metrics
Aggregated GQ, DP

Bronze · Spark · Glow · External CLI · Silver Table · Gold Table · Quarantine

Executor Cache Warming

Prior to executing Catalyst logical plans, the driver must securely mount the reference genome via Unity Catalog. Because JNI-bound libraries (e.g., htsjdk) strictly require POSIX-compliant file handles, we must bypass DBFS FUSE latency by distributing the FASTA indices directly to executor NVMe caches.

1. Unity Catalog Provisioning

The hg38.fa payload is materialized in UC Volumes: /Volumes/.../hg38.fa, providing a governed, non-HDFS credentialed mount point.

2. Byte-Offset Indexing (.fai)

Glow normalizers require strict byte-offset mappings. A synchronous Python build_fai() function computes the line bases and offsets to serialize a local .fai.

3. mapPartitions Distribution

A dummy RDD triggers a cluster-wide broadcast, coercing all JVM executors to execute shutil.copy to local /tmp, eliminating JNI file-lock exceptions.

- **Databricks runtime:** 16.4 LTS (includes Apache Spark 3.5.2, Scala 2.12
- **Library:** io.projectglow:glow-spark3_2.12:2.0.0
- **Preferred worker type:** rd-fleet.xlarge

VCF Ingestion & Schema Binding

Step 4: JVM Spark Registration

To expose Glow's C++ vectorized functions to the Spark SQL engine:

- Inject via %pip install glow.py
- Bind the UDFs into the Catalyst registry via glow.register(spark)

Step 5: Spark Read Execution

Ingesting VCFs leveraging the native Glow data source:

- spark.read.format("vcf") interprets the headers.
- *Crucial*: Explicit StructType injection prevents expensive schema inference and CANNOT_DETERMINE_TYPE panics on deeply nested arrays.

Core DAG Operations

Spark orchestrates the DAG to execute Glow transformations. Complex arrays of ALTs are exploded into flat memory structures. The local /tmp FASTA allows parsimonious left-alignment at scale, while the Pipe operator handles standard I/O streams to external bash subprocesses.

6. split_multiallelics

Translates multi-ALT vectors into a flattened DataFrame space. Resolves cardinality mismatch but induces high memory pressure during the shuffle phase.

7. normalize_variants

Executes deterministic left-alignment of INDELs. Requires reference_genome_path pointing to the warmed local NVMe cache (/tmp/ref_genome) for random access.

8. Glow Pipe Operator

Streams DataFrame memory directly through STDIN/STDOUT to external CLI processes (e.g., VEP, Mutect) utilizing input_formatter="vcf", recapturing output as native rows.

Data Model Flattening & Delta Lake

Step 9: Array Destructuring

Using F.explode_outer() and recursive struct traversal, highly-nested Genotypes and CSQ payloads are pivoted into structured relational formats:

- **variants_flat:** Hash-keyed via chrom:pos:ref:alt
- **variant_calls:** Destructured PL, AD, and DP arrays
- **variant_consequences_flat:** Exploded CSQ structs

Step 10: Delta Parquet Commits

saveAsTable("users...variants_flat") initiates the write phase:

- Stored as Z-Ordered, Columnar Parquet
- ACID guarantees via the _delta_log transaction ledger
- autoMerge.enabled allows schema evolution on nested VCF INFO flags

Logical Execution Plan Summary

1. RDD Ingest

Driver analyzes DBFS volumes; Executors perform parallel VCF block ingestion.

2. Explode & Align

Catalyst optimization applies split_multiallelics and JNI-backed normalization.

3. Pipe STDOUT

DataFrames stream through external OS processes; vcf_pipe_quarantine traps non-zero exits.

4. Delta Commit

Structs destructured to flat primitives; atomic transactional commits to Object Storage.

Distributed Systems Obstacles & Resolutions

Architectural Obstacle	Implemented Technical Resolution
VCF CANNOT_DETERMINE_TYPE Exceptions	Injected immutable StructType schemas directly into the DataFrame read protocol.
JNI/htsjdk FUSE Mount Latency & Locks	Deployed mapPartitions dummy job to force local NVMe caching of FASTA on all nodes.
High Shuffling on Multi-allelics	Utilized native Glow split_multiallelics to enforce row-level atomicity pre-shuffle.
External Subprocess Panics (OOM/Crash)	Enforced quarantine_flavor="delta" in Pipe to catch and log failed STDOUT serialization.
Schema Evolution on VCF INFO keys	Enabled spark.databricks.delta.schema.autoMerge.enabled = true.

5. Key Business Benefits & ROI

- **Infinite Scalability:** The pipeline eliminates the memory limits and slow processing times associated with legacy tools, easily scaling to handle large population cohorts.
- **Data Reliability:** By writing to Delta Lake (Parquet), the pipeline guarantees ACID compliance (transactional integrity) and prevents data corruption during complex transformations.
- **Faster Time-to-Insight:** Complex, deeply nested genomic arrays are flattened into standard relational formats. This enables immediate, massive parallel querying using standard ANSI SQL, drastically reducing the time it takes analysts to extract insights.
- **Fault Tolerance:** Built-in quarantine mechanisms (`vcf_pipe_quarantine`) trap and log malformed records without crashing the entire pipeline run, ensuring continuous operational uptime.

6. Conclusion

By adopting Databricks and Glow, NYU Langone has built an enterprise-grade data lakehouse capable of merging complex genomic variant data with clinical EHR records. While traditional tools remain essential for upstream variant discovery, this new distributed framework orchestrates the heavy lifting—providing a robust, scalable, and SQL-friendly foundation for advanced bioinformatics and clinical analytics.

Appendix

Genomic Framework Architectures: Capability & Execution Comparisons

Genomics Data Processing Frameworks

Capability	Glow (Apache Spark)	bcftools / htslib	GATK (Broad Inst.)	Hail (Spark/Batch)
Execution Paradigm	Distributed In-Memory DAG	Single-Node Sequential	Single-Node / WDL	Distributed Matrix/Table
Vectorization	✓ Photon / Spark SQL	✗ Iterative C	✗ Iterative Java	✓ Custom C++ JVM
Storage Primitives	✓ Parquet / Delta Lake	✗ Flat VCF/BCF	✗ Flat VCF/BAM	✓ BlockMatrix / Parquet
Quarantine / Fault Tolerance	✓ Native Delta Trapping	✗ Fails entire process	✗ Standard Error Logs	✓ Job Retries

Architectural Positioning: Glow vs bcftools

Glow (Databricks)

Mandatory for Terabyte/Petabyte scale. Shifts processing from single-node bottlenecks to distributed MapReduce/Spark paradigms. Essential when the target sink is a heavily partitioned Delta Lake architecture for scalable BI integration.

bcftools / htslib

Retains superiority for ad-hoc, localized stream manipulation (C-optimized). Best employed inside isolated Docker containers where data volumes do not necessitate cluster computing overhead.

Positioning: GATK, VEP & Hail

GATK (HaplotypeCaller)

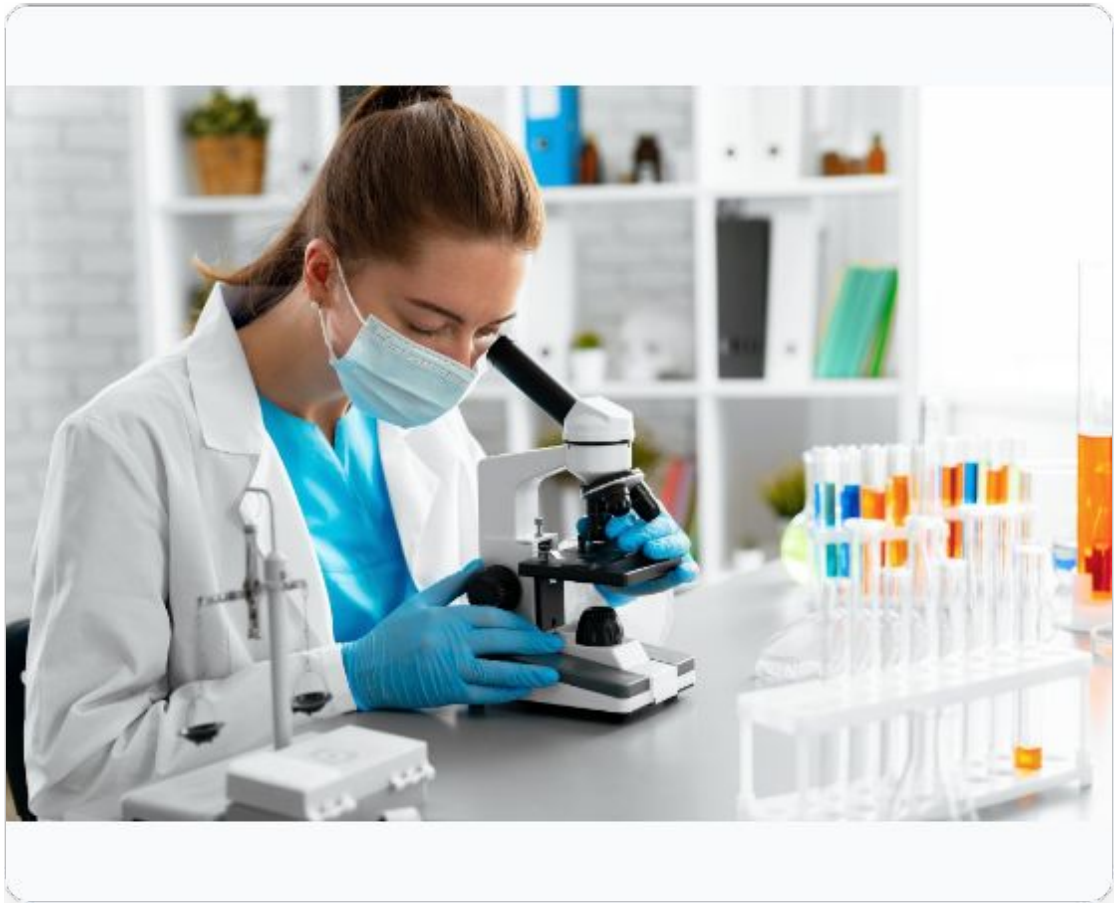
Utilized strictly upstream in the pipeline architecture for algorithm-heavy variant calling (HMMs, BQSR). Not designed for downstream analytical ETL.

VEP / ANNOVAR

CLI-based annotation engines. Designed to be *wrapped* by Glow's Pipe transformer to achieve distributed execution across a Spark cluster.

Hail

Highly specialized Spark library focusing on population-scale matrix algebra (GWAS, PCA) rather than general-purpose ETL and pipeline orchestration.



Macro-Pipeline Topology

1. Upstream (GATK / BWA-MEM) FASTQ sequences are mapped, aligned (BAM), and evaluated via HaplotypeCaller, generating the raw source VCF payload.

2. Glow Orchestration (Compute) Spark reads the distributed VCF blocks, executing Catalyst-optimized split_multiallelics and JNI-driven left-alignment.

3. Subprocess Offload (VEP) Glow partitions DataFrame vectors through OS Pipes to external binaries for distributed functional annotation.

4. Analytics Sink (Delta Lake) Vectors are recursively flattened into optimized columnar storage, ready for massively parallel SQL querying via Databricks SQL.

Architectural Trade-offs

Glow (Spark + Delta)

- ✓ Horizontal scaling via RDD partitioning
- ✓ Native integration with Parquet / Delta optimizations (Z-Order)
- ✓ Trivial querying via ANSI SQL
- ✗ Mandates high-overhead JVM / Spark Cluster compute
- ✗ Annotation requires complex OS-level pipe integrations

Legacy POSIX Frameworks

- ✓ **GATK:** Unmatched statistical accuracy for discovery
- ✓ **VEP:** Turn-key biological consequence databases
- ✓ **bcftools:** Lowest possible overhead for small jobs
- ✗ **Constraint:** Vertical scaling limits (OOMs), slow single-thread disk I/O, and incompatibility with modern Data Lakehouse BI patterns.